Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kĩ thuật Máy tính



THỰC HÀNH KIẾN TRÚC MÁY TÍNH
(CO2008)

Báo cáo Lab 1

Arithmetic instructions
(Các lệnh số học)

Giảng viên hướng dẫn: Tôn Huỳnh Long
Nhóm: L01
Sinh viên: Đặng Minh Triết - 2510249

Thành phố Hồ Chí Minh, Tháng 10/2026

Mục lục

1 Bài 1 – In "Hello, <name>!" 2
1.1 Đề bài . . . 2
1.2 Ý tưởng . . . 2
1.3 Mã nguồn . . . 3
1.4 Kết quả chạy trên MARS . . . 4

2 Bài 2 – Nhập mảng 5 phần tử và in theo chỉ số 4
2.1 Đề bài . . . 4
2.2 Ý tưởng . . . 4
2.3 Mã nguồn . . . 5
2.4 Kết quả chạy trên MARS . . . 6

3 Bài 3 – Tính biểu thức F 6
3.1 Đề bài . . . 6
3.2 Ý tưởng . . . 7
3.3 Mã nguồn . . . 8
3.4 Kết quả chạy trên MARS . . . 9

4 Bài 4 – In dạng nhị phân 4 bit 9
4.1 Đề bài . . . 9
4.2 Ý tưởng . . . 10
4.3 Mã nguồn . . . 10
4.4 Kết quả chạy trên MARS . . . 12

5 Bài 5 – Đảo ngược mảng 10 phần tử 12
5.1 Đề bài . . . 12
5.2 Ý tưởng . . . 12
5.3 Mã nguồn . . . 13
5.4 Kết quả chạy trên MARS . . . 14

6 Bài 6 – Dung lượng khung hình và thời gian truyền 14
6.1 Đề bài . . . 14
6.2 Lời giải . . . 14

7 Bài 7 – Hiệu năng bộ xử lý 14
7.1 Đề bài . . . 14
7.2 Lời giải . . . 15

Tài liệu tham khảo 17

1 Bài 1 – In "Hello, <name>!"

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/file_asm/ex1_lab1.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/video/video_ex1_lab1.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1tLy7BGXtTx7sZRRVEhPpJ_5Q2frk9xN_/view?usp=drive_link

1.1 Đề bài

Viết một chương trình MIPS nhận vào một tên dưới dạng chuỗi và in ra `"Hello <name>!"`. Ví dụ, nếu đầu vào là `John` thì đầu ra phải là `Hello, John!`

1.2 Ý tưởng

1. Đọc tên bằng `syscall` 8 vào bộ đệm `name` 50 byte (`$a1` = 50, đọc tối đa 49 kí tự). Đề bài không yêu cầu lời nhắc nên chương trình đọc tên ngay.
2. **Vấn đề:** `syscall` 8 tự thêm kí tự xuống dòng `'\n'` (mã ASCII 10) vào sau tên, ví dụ nhập `John` thì bộ nhớ chứa `"John\n\0"`. Nếu in thẳng thì dấu `!` bị đẩy xuống dòng dưới.
3. **Cách giải:** duyệt chuỗi từng kí tự một (giống ví dụ *String Copy* trong chương 2). `$s0` giữ địa chỉ kí tự đang xét; mỗi vòng lặp:
   - Đọc kí tự bằng `lbu $t0, 0($s0)`.
   - Nếu kí tự là `'\0'` (`beq` với `$0`, thanh ghi luôn bằng 0, đúng bằng mã ASCII của `'\0'`): chuỗi đã hết mà không có `'\n'` (trường hợp nhập đủ 49 kí tự), nhảy thẳng tới nhãn `print`.
   - Nếu kí tự là `'\n'` (`beq` với `$t1` = 10): nhảy tới nhãn `change`, ghi `'\0'` vào vị trí đó bằng `sb $zero, 0($s0)` (`$zero` chính là `$0`). Chuỗi kết thúc ngay sau tên.
   - Nếu không: tăng `$s0` thêm 1 (`addi`) để sang kí tự kế tiếp, rồi quay lại đầu vòng lặp (`j loop`).
4. In lần lượt `"Hello, "`, `name` và `"!"` bằng `syscall` 4.

Bảng 1: Các vòng lặp khi nhập `John` (bộ nhớ chứa `"John\n\0"`)

Vòng	Vị trí	Kí tự	Xử lý
1	`name[0]`	`'J'` (74)	không phải `'\0'`/`'\n'` ⇒ sang kí tự kế
2	`name[1]`	`'o'` (111)	không phải `'\0'`/`'\n'` ⇒ sang kí tự kế
3	`name[2]`	`'h'` (104)	không phải `'\0'`/`'\n'` ⇒ sang kí tự kế
4	`name[3]`	`'n'` (110)	không phải `'\0'`/`'\n'` ⇒ sang kí tự kế
5	`name[4]`	`'\n'` (10)	ghi `'\0'` ⇒ chuỗi thành `"John"`, đi in

1.3 Mã nguồn

```
.data:
hello: .asciiz "Hello, "
name: .space 50 # Ten (toi da 50 ki tu)
exclamation_mark: .asciiz "!"

.text
.globl main
main:
        # Nhap chuoi
        # syscall 8 se bi van de tu dong them \n sau name VD nhu "John\n"
        li $v0, 8 # Doc ten va luu vao name
        la $a0, name
        li $a1, 50 # toi da 50 ki tu
        syscall

        # Xoa \n trong name
        la $s0, name
        li $t1, 10 # ma ascii cua "\n"
loop:
        lbu $t0, 0($s0)
        beq $t0, $0, print # Neu ki tu la "\0" -> chuyen qua print ($0 = \0)
        beq $t0, $t1, change # Neu ki tu la "\n" -> chuyen qua change
        addi $s0, $s0, 1
        j loop

change: # Doi \n thanh \0
        sb $zero, 0($s0)

print: # In name
        li $v0, 4 # In chu "Hello, "
        la $a0, hello
        syscall

        la $a0, name # In name
        syscall

        la $a0, exclamation_mark #In dau "!"
        syscall

        li $v0, 10 #Ket thuc
        syscall
```

1.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề (dòng đầu là tên được nhập vào, dòng sau là kết quả):

```
John
Hello, John!
```

2 Bài 2 – Nhập mảng 5 phần tử và in theo chỉ số

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/file_asm/ex2_lab1.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/video/video_ex2_lab1.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1P2p8AoOpPVansc_Cr52bpzdvOR7om6Vm/view?usp=drive_link

2.1 Đề bài

Viết một chương trình MIPS cho phép người dùng nhập trước một mảng gồm 5 phần tử. Các lời nhắc phải là: `"Please input element 0:"`, `"Please input element 1:"`, v.v. Sau đó in ra giá trị của phần tử do người dùng chọn (giả sử người dùng chỉ nhập từ 0 đến 4). Lời nhắc cho bước này là: `"Please enter index:"`.
Ví dụ, một mảng có thể có các phần tử như 5, 10, 15, 20, 25. Nếu người dùng nhập 1 thì kết quả hiển thị trên màn hình phải là 10. Nếu người dùng nhập 4 thì kết quả phải là 25.

2.2 Ý tưởng

1. Dành mảng `array` 20 byte bằng `.space 20` (5 phần tử, mỗi phần tử là 1 word 4 byte). `array` được khai báo đầu tiên trong `.data` nên nằm ở địa chỉ `0x10010000`, chia hết cho 4, dùng được `lw`/`sw`.
2. **Nhập mảng bằng vòng lặp** (nhãn `input`): `$t0` là biến đếm (bắt đầu từ 0), `$t1` là con trỏ trỏ tới phần tử đang nhập (bắt đầu từ `array`). Mỗi vòng lặp:
   - In lời nhắc gồm 3 phần: chuỗi `"Please input element "`, số thứ tự `$t0` (`syscall` 1) và `":"`.
   - Đọc số nguyên bằng `syscall` 5 (kết quả trong `$v0`) và lưu vào phần tử hiện tại bằng `sw $v0, 0($t1)`.
   - Tăng biến đếm thêm 1 và tăng con trỏ thêm 4 byte để trỏ sang phần tử kế tiếp (`addi`).
   - Nếu đã nhập đủ 5 phần tử (`beq $t0, 5, index`) thì sang phần nhập chỉ số; nếu chưa thì quay lại đầu vòng lặp (`j input`).
3. **Lấy phần tử theo chỉ số** (nhãn `index`): đọc chỉ số vào `$t0`. Địa chỉ phần tử là $\texttt{array} + 4 \times \texttt{index}$ (mỗi phần tử 4 byte), tính bằng `mul $t0, $t0, 4` rồi `add` với địa chỉ `array`. Đọc phần tử bằng `lw` và in ra bằng `syscall` 1.
4. `beq $t0, 5, index` và `mul $t0, $t0, 4` có toán hạng là hằng số nên là lệnh giả: MARS tự nạp hằng số vào thanh ghi `$at` trước (ví dụ `addi $at, $0, 5`), rồi mới thực hiện `beq`/`mul` với `$at`.

Bảng 2: Địa chỉ các phần tử của `array` (bắt đầu tại `0x10010000`)

Chỉ số	$4 \times$ chỉ số	Địa chỉ	Giá trị (ví dụ)
0	0	0x10010000	5
1	4	0x10010004	10
2	8	0x10010008	15
3	12	0x1001000C	20
4	16	0x10010010	25

2.3 Mã nguồn

```
.data
array: .space 20 # Mang chua 5 element
prompt_input: .asciiz "Please input element "
colon: .asciiz ": "
prompt_index: .asciiz "Please enter index: "

.text
.globl main
main:
        li $t0, 0 #Bien dem
        la $t1, array

input: # Nhap 5 element
        li $v0, 4 # In "Please input element "
        la $a0, prompt_input
        syscall

        li $v0, 1 # In bien dem
        move $a0, $t0
        syscall

        li $v0, 4 # In dau ":"
        la $a0, colon
        syscall

        li $v0, 5 # Doc so (element)
        syscall

        # Luu element vao array
        sw $v0, 0($t1)

        addi $t0, $t0, 1 #Cap nhat bien dem
        addi $t1, $t1, 4 #Cap nhat dia chi array
        beq $t0, 5, index #Du 5 element -> chuyen qua index
        j input

index: # Nhap index
        li $v0, 4 # In "Please enter index:"
        la $a0, prompt_index
        syscall

        li $v0, 5 # Doc index
        syscall

```

```
45          # Tra ve element tuong ung
46          move $t0, $v0 # t0 tro thanh index
47          la $t1, array # Ve lai dia chi co so cua array
48          mul $t0, $t0, 4 # index*4
49          add $t1, $t1, $t0 # &array[4*index]
50          lw $t2, 0($t1)
51
52          li $v0, 1 # In element
53          move $a0, $t2
54          syscall
55
56          li $v0, 10 #Ket thuc
57          syscall
```

2.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề: mảng 5, 10, 15, 20, 25, lần lượt nhập chỉ số 1 và 4:

```
Please input element 0: 5
Please input element 1: 10
Please input element 2: 15
Please input element 3: 20
Please input element 4: 25
Please enter index: 1
10

Please input element 0: 5
Please input element 1: 10
Please input element 2: 15
Please input element 3: 20
Please input element 4: 25
Please enter index: 4
25
```

3 Bài 3 – Tính biểu thức F

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/file_asm/ex3_lab1.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/video/video_ex3_lab1.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1v_fMkfPdT52sFFBcKWMghjrk4F4ce2xQ/view?usp=drive_link

3.1 Đề bài

Viết một chương trình MIPS in ra kết quả của F:

$$F = \frac{(a+10) \times (b-d) \times (c-2\times a)}{a+b+c}$$

trong đó a, b, c và d là các số nguyên do người dùng nhập vào. Lưu ý rằng a, b, c và d được nhập theo đúng thứ tự này. Ngoài ra, giả sử `"a+b+c"` khác 0. Kết quả của phép chia phải in ra cả thương và số dư.
Các lời nhắc nhập phải có dạng: `"Insert a: "`, `"Insert b: "`, v.v.
Ví dụ, nếu đầu vào là 3, 4, 5, 6 thì đầu ra phải là: `F = 2, remainder 2`.

3.2 Ý tưởng

1. **Nhập a, b, c, d theo đúng thứ tự:** với mỗi số, in lời nhắc `"Insert a: "`, ... bằng `syscall` 4 rồi đọc số nguyên bằng `syscall` 5. Kết quả nằm trong `$v0` nên được chép ngay sang `$t0`, `$t1`, `$t2`, `$t3` bằng `move`, vì lần gọi `syscall` kế tiếp sẽ ghi đè `$v0`.
2. **Tính tử số** $(a+10) \times (b-d) \times (c-2a)$:
   - `$t4` $= a+10$ (`addi`), `$t5` $= b-d$ (`sub`);
   - `$t6` $= 2a = a+a$ (`add`), `$t7` $= c-2a$ (`sub`);
   - nhân dồn bằng hai lệnh `mul`: `$s0` = `$t4` $\times$ `$t5` $\times$ `$t7`.
3. **Tính mẫu số** `$s1` $= a+b+c$ bằng hai lệnh `add`.
4. **Chia:** lệnh `div $s0, $s1` đặt **thương** vào thanh ghi `LO` và **số dư** vào `HI`. Lấy ra bằng `mflo $s2` và `mfhi $s3`, rồi in theo định dạng `F = <thương>, remainder <số dư>`.

Bảng 3: Các thanh ghi dùng trong bài 3

Thanh ghi	Nội dung	Thanh ghi	Nội dung
`$t0`, `$t1`, `$t2`, `$t3`	a, b, c, d	`$s0`	tử số
`$t4`	$a+10$	`$s1`	mẫu số $a+b+c$
`$t5`	$b-d$	`$s2`	thương F
`$t6`, `$t7`	$2a$, $c-2a$	`$s3`	số dư

Kiểm tra với ví dụ của đề ($a=3, b=4, c=5, d=6$):

$$\frac{(3+10)(4-6)(5-2\cdot 3)}{3+4+5} = \frac{13\cdot(-2)\cdot(-1)}{12} = \frac{26}{12} \Rightarrow \text{thương } 2,\ \text{dư } 2.$$

Lưu ý:

- Lệnh `div` của MIPS chia làm tròn về 0, số dư cùng dấu với số bị chia. Ví dụ $a=1, b=2, c=3, d=4$ cho $-22/6$: thương -3, dư -4 (vì $-3\cdot 6+(-4)=-22$).
- `mul` chỉ giữ 32 bit thấp của tích, nên kết quả đúng khi tử số nằm trong phạm vi số nguyên 32 bit có dấu.

3.3 Mã nguồn

```
.data
prompt_a: .asciiz "Insert a: "
prompt_b: .asciiz "Insert b: "
prompt_c: .asciiz "Insert c: "
prompt_d: .asciiz "Insert d: "
prompt_F: .asciiz "F = "
prompt_remainder: .asciiz ", remainder "

.text
.globl main
main:
        # Lay gia tri a,b,c,d
        # Lay gia tri a
        li $v0, 4
        la $a0, prompt_a
        syscall

        li $v0, 5
        syscall

        move $t0, $v0 # $t0 = a

        # Lay gia tri b
        li $v0, 4
        la $a0, prompt_b
        syscall

        li $v0, 5
        syscall

        move $t1, $v0 # $t1 = b

        # Lay gia tri c
        li $v0, 4
        la $a0, prompt_c
        syscall

        li $v0, 5
        syscall

        move $t2, $v0 # $t2 = c

        # Lay gia tri d
        li $v0, 4
        la $a0, prompt_d
        syscall

        li $v0, 5
        syscall

        move $t3, $v0 # $t3 = d

        # Tinh F va remainder
        # Tinh (a+10)x(b-d)x(c-2xa)
        addi $t4, $t0, 10 # $t4 = (a + 10)
        sub $t5, $t1, $t3 # $t5 = (b - d)
        add $t6, $t0, $t0
        sub $t7, $t2, $t6 # $t7 = (c - 2 x a)
```

```
        mul $s0, $t4, $t5
        mul $s0, $s0, $t7 # $s0 = (a+10)x(b-d)x(c-2xa)
        # Tinh (a+b+c)
        add $s1, $t0, $t1
        add $s1, $s1, $t2 # $s1 = (a+b+c)
        # Phep chia
        div $s0, $s1
        mflo $s2 # $s2 = F
        mfhi $s3 # $s3 = remainder

        li $v0, 4
        la $a0, prompt_F
        syscall

        li $v0, 1 # In F
        move $a0, $s2
        syscall

        li $v0, 4
        la $a0, prompt_remainder
        syscall

        li $v0, 1 # In remainder
        move $a0, $s3
        syscall

        li $v0, 10 #Ket thuc
        syscall
```

3.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề $(a, b, c, d = 3, 4, 5, 6)$:

```
Insert a: 3
Insert b: 4
Insert c: 5
Insert d: 6
F = 2, remainder 2
```

4 Bài 4 – In dạng nhị phân 4 bit

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/file_asm/ex4_lab1.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/video/video_ex4_lab1.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1KGB0CBTfQTg5UgRUQdjmOeSvJGzzGNW-/view?usp=drive_link

4.1 Đề bài

Viết một chương trình MIPS in ra giá trị nhị phân 4 bit của một số thập phân. Số này do người dùng chọn.

Đầu vào chỉ được giới hạn là một số nguyên dương nhỏ hơn 16. Lời nhắc nhập phải là: `"Please enter a positive integer less than 16: "`.
Ví dụ, nếu số thập phân là 10 thì đầu ra phải là `Its binary form is: 1010`.

4.2 Ý tưởng

Mỗi số $0 \le n < 16$ viết được duy nhất dưới dạng

$$n = b_3 \cdot 8 + b_2 \cdot 4 + b_1 \cdot 2 + b_0, \qquad b_i \in \{0, 1\}.$$

Không dùng lệnh dịch bit hay lệnh logic, ta tách bit bằng cách **chia liên tiếp cho 2** (`div`, `mflo`, `mfhi`), giống cách đổi số thập phân sang nhị phân bằng tay:

1. Đặt số chia `$t0` $= 2$ và đọc n vào `$t1`.
2. Chia `$t1` cho 2: **số dư** (`mfhi`) là một bit, **thương** (`mflo`) được ghi đè lại vào `$t1` để chia tiếp ở lần sau.
3. Lặp lại 4 lần (viết lần lượt, không dùng vòng lặp): các số dư lần lượt là b_0, b_1, b_2, b_3, cất vào `$t2`, `$t3`, `$t4`, `$t5`.
4. Bit tìm được theo thứ tự từ thấp lên cao, nên in theo **thứ tự ngược lại**: b_3, b_2, b_1, b_0 (`$t5`, `$t4`, `$t3`, `$t2`) bằng `syscall` 1. Mỗi bit được in riêng nên các số 0 ở đầu vẫn được giữ, ví dụ $n = 1$ in ra `0001`.

Bảng 4: Các bước tách bit với $n = 10$

Lần chia	Phép chia	Thương (chia tiếp)	Số dư (bit)	Thanh ghi
1	$10 \div 2$	5	$b_0 = 0$	`$t2`
2	$5 \div 2$	2	$b_1 = 1$	`$t3`
3	$2 \div 2$	1	$b_2 = 0$	`$t4`
4	$1 \div 2$	0	$b_3 = 1$	`$t5`
In theo thứ tự $b_3 b_2 b_1 b_0$: 1010				

4.3 Mã nguồn

```
.data
prompt_input: .asciiz "Please enter a positive integer less than 16: "
prompt_binary: .asciiz "Its binary form is: "

.text
.globl main
main: # Doc so
        li $t0, 2

        li $v0, 4
        la $a0, prompt_input
        syscall

        li $v0, 5 # Doc so
```

```
15          syscall
16
17          move $t1, $v0
18
19          # Tim 4 bit
20          div $t1, $t0
21          mflo $t1 # thuong
22          mfhi $t2 # $t2 = bit-0
23
24          div $t1, $t0
25          mflo $t1 # thuong
26          mfhi $t3 # $t3 = bit-1
27
28          div $t1, $t0
29          mflo $t1 # thuong
30          mfhi $t4 # $t4 = bit-2
31
32          div $t1, $t0
33          mflo $t1 # thuong
34          mfhi $t5 # $t5 = bit-3
35
36          # In so binary
37          li $v0, 4
38          la $a0, prompt_binary
39          syscall
40
41          li $v0, 1
42          move $a0, $t5 #bit-3
43          syscall
44
45          li $v0, 1
46          move $a0, $t4 #bit-2
47          syscall
48
49          li $v0, 1
50          move $a0, $t3 #bit-1
51          syscall
52
53          li $v0, 1
54          move $a0, $t2 #bit-0
55          syscall
56
57          li $v0, 10 #Ket thuc
58          syscall
```

4.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề ($n = 10$):

```
Please enter a positive integer less than 16: 10
Its binary form is: 1010
```

5 Bài 5 – Đảo ngược mảng 10 phần tử

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/file_asm/ex5_lab1.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_1/video/video_ex5_lab1.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1DT-5NySeJlYdpiRQN41K9dITaMOppz-x/view?usp=drive_link

5.1 Đề bài

Viết một chương trình MIPS in ra các phần tử của một mảng 10 số nguyên do bạn tự chọn theo thứ tự đảo ngược.
Ví dụ, nếu mảng định sẵn của bạn chứa 1, 2, 5, 8, 12, 44, 3, 9, 0, 10 thì kết quả in ra phải là 10, 0, 9, 3, 44, 12, 8, 5, 2, 1.

5.2 Ý tưởng

1. Khai báo mảng `array` bằng `.word` với 10 giá trị như ví dụ của đề. Mỗi phần tử chiếm 4 byte, nên phần tử có chỉ số k nằm ở địa chỉ $\texttt{array} + 4k$; phần tử cuối (chỉ số 9) nằm ở $\texttt{array} + 36$.

2. **Bắt đầu từ phần tử cuối:** $\texttt{\$t0} = 9$ là chỉ số phần tử đang in, $\texttt{\$t1} = \texttt{array} + 36$ là địa chỉ của nó.

3. **Vòng lặp in ngược** (nhãn `print`), mỗi vòng:

   - Đọc phần tử bằng `lw $t2, 0($t1)` và in ra bằng `syscall` 1.
   - Nếu vừa in xong phần tử đầu tiên (`beq $t0, 0, end`) thì kết thúc.
   - Nếu chưa, in dấu phân cách `", "`, giảm chỉ số đi 1 và lùi địa chỉ về 4 byte (`addi` với -1 và -4), rồi quay lại đầu vòng lặp (`j print`).

4. Việc kiểm tra điều kiện dừng đặt **sau khi in phần tử** nhưng **trước khi in dấu phẩy**, nên dấu `", "` chỉ xuất hiện giữa các phần tử, không bị thừa ở cuối dòng.

Bảng 5: Các vòng lặp in ngược mảng (bắt đầu tại `array` = `0x10010000`)

`$t0` (chỉ số)	`$t1` (địa chỉ)	Phần tử in ra	Sau khi in
9	0x10010024	10	in ", ", lùi về chỉ số 8
8	0x10010020	0	in ", ", lùi về chỉ số 7
		⋮	
1	0x10010004	2	in ", ", lùi về chỉ số 0
0	0x10010000	1	$t0 = 0 ⇒ kết thúc

5.3 Mã nguồn

```
.data
array: .word 1, 2, 5, 8, 12, 44, 3, 9, 0, 10 #mang cho truoc
comma: .asciiz ", "

.text
.globl main
main:
        li $t0, 9 #index chi so cuoi
        la $t1, array
        addi $t1, $t1, 36

print:
        li $v0, 1
        lw $t2, 0($t1)
        move $a0, $t2
        syscall

        beq $t0, 0, end # het so de in -> end

        li $v0, 4
        la $a0, comma
        syscall

        addi $t0, $t0, -1
        addi $t1, $t1, -4
        j print

end:
        li $v0, 10
        syscall
```

5.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề (mảng cho trước `1, 2, 5, 8, 12, 44, 3, 9, 0, 10`):

```
10, 0, 9, 3, 44, 12, 8, 5, 2, 1
```

6 Bài 6 – Dung lượng khung hình và thời gian truyền

6.1 Đề bài

Cấu hình của hệ thống hiển thị màu như sau:

- Mỗi màu được biểu diễn bằng 8 bit, mỗi điểm ảnh (pixel) gồm 3 màu (đỏ, xanh lá, xanh dương).
- Độ phân giải 1280x1024.

a. Xác định dung lượng tối thiểu của mỗi khung hình (frame).

b. Xác định thời gian truyền khung hình đó, biết tốc độ mạng là 100 Mbit/s.

6.2 Lời giải

a. Dung lượng mỗi khung hình

Mỗi điểm ảnh cần $3 \times 8 = 24$ bit. Số điểm ảnh trong một khung hình là $1280 \times 1024 = 1\,310\,720$. Do đó:

$$\begin{aligned}\text{Dung lượng} &= 1280 \times 1024 \times 24 = 31\,457\,280 \text{ bit} \\ &= \frac{31\,457\,280}{8} = 3\,932\,160 \text{ byte} \\ &= \frac{3\,932\,160}{1024} = 3840 \text{ KiB} = \frac{3840}{1024} = 3{,}75 \text{ MiB } (\approx 3{,}93 \text{ MB}).\end{aligned}$$

Đáp số: mỗi khung hình cần tối thiểu $31\,457\,280$ bit $= 3\,932\,160$ byte $= 3{,}75$ MiB.

b. Thời gian truyền một khung hình

Tốc độ mạng 100 Mbit/s $= 100 \times 10^6$ bit/s, nên

$$t = \frac{\text{Dung lượng}}{\text{Tốc độ}} = \frac{31\,457\,280 \text{ bit}}{100 \times 10^6 \text{ bit/s}} \approx 0{,}3146 \text{ s} \approx 314{,}6 \text{ ms}.$$

Đáp số: $t \approx 0{,}315$ s.

7 Bài 7 – Hiệu năng bộ xử lý

7.1 Đề bài

Ba bộ xử lý thực thi cùng một tập lệnh, với tần số và CPI như sau:

- Bộ xử lý P1: tần số xung nhịp 3 GHz, CPI 1,5

- Bộ xử lý P2: tần số xung nhịp 2,5 GHz, CPI 1,0
- Bộ xử lý P3: tần số xung nhịp 4 GHz, CPI 2,2

a. Bộ xử lý nào có hiệu năng cao nhất tính theo số lệnh mỗi giây (IPS)?

b. Nếu một bộ xử lý thực thi một chương trình trong 10 giây, hãy tìm số lệnh và tổng số chu kỳ đã thực thi.

c. Nếu thời gian thực thi chương trình giảm 30% thì CPI tăng 20%. Tần số của hệ thống phải là bao nhiêu để đạt được thời gian đó?

7.2 Lời giải

Sử dụng các công thức:

$$T_{CPU} = \frac{IC \times CPI}{f} \qquad \Longrightarrow \qquad IPS = \frac{IC}{T_{CPU}} = \frac{f}{CPI},$$

$$\text{Số chu kỳ} = f \times T_{CPU}, \qquad IC = \frac{\text{Số chu kỳ}}{CPI},$$

trong đó IC là số lệnh thực thi, f là tần số xung nhịp.

a. So sánh IPS

$$IPS_{P1} = \frac{3 \times 10^9}{1{,}5} = 2{,}0 \times 10^9 \text{ lệnh/s}$$
$$IPS_{P2} = \frac{2{,}5 \times 10^9}{1{,}0} = 2{,}5 \times 10^9 \text{ lệnh/s}$$
$$IPS_{P3} = \frac{4 \times 10^9}{2{,}2} \approx 1{,}82 \times 10^9 \text{ lệnh/s}$$

Đáp số: P2 có hiệu năng cao nhất ($2{,}5 \times 10^9$ lệnh/s).

b. Số chu kỳ và số lệnh trong 10 giây

Số chu kỳ $= f \times 10$; số lệnh $IC = \dfrac{\text{Số chu kỳ}}{CPI} = IPS \times 10$.

Bảng 6: Số chu kỳ và số lệnh khi chạy chương trình trong 10 giây

Bộ xử lý	f	CPI	Số chu kỳ	Số lệnh IC
P1	3 GHz	1,5	$3{,}0 \times 10^{10}$	$2{,}0 \times 10^{10}$
P2	2,5 GHz	1,0	$2{,}5 \times 10^{10}$	$2{,}5 \times 10^{10}$
P3	4 GHz	2,2	$4{,}0 \times 10^{10}$	$\approx 1{,}82 \times 10^{10}$

c. Tần số mới

Chương trình (số lệnh IC) không đổi; thời gian mới $T' = (1 - 0{,}3)\,T = 0{,}7\,T$; CPI mới $CPI' = 1{,}2 \times CPI$. Từ $T = \dfrac{IC \times CPI}{f}$:

$$f' = \frac{IC \times CPI'}{T'} = \frac{IC \times 1{,}2\,CPI}{0{,}7\,T} = \frac{1{,}2}{0{,}7} \cdot \frac{IC \times CPI}{T} = \frac{12}{7} f.$$

Vậy tần số phải tăng khoảng $71{,}4\%$. Áp dụng cho từng bộ xử lý:

$$f'_{P1} = \tfrac{12}{7} \times 3 \approx 5{,}14 \text{ GHz}, \qquad f'_{P2} = \tfrac{12}{7} \times 2{,}5 \approx 4{,}29 \text{ GHz}, \qquad f'_{P3} = \tfrac{12}{7} \times 4 \approx 6{,}86 \text{ GHz}.$$

Tài liệu tham khảo

[1] Pete Sanderson, Kenneth Vollmar. *MARS (MIPS Assembler and Runtime Simulator), phiên bản 4.5.* Truy cập vào 09/10/2026. https://dpetersanderson.github.io/

[2] David A. Patterson, John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface, MIPS Edition.* Morgan Kaufmann.

[3] Khoa Khoa học và Kĩ thuật Máy tính, Trường Đại học Bách khoa – ĐHQG-HCM. *Computer Architecture Lab (CO2008) – Lab 1: Arithmetic instructions.* 2025.